

データベース演習

第9回：アプリケーション開発フレームワークとデータベース

鈴木源吾

前回の復習

1. 導入と環境の準備

- 演習に使うPython環境のセットアップ
- セルでのSQL実行：JupySQL

2. Pythonでデータベースアクセス：概要と検索

今回のトピック

1. Pythonからのデータ作成 (INSERT)
2. アプリケーション開発フレームワークとデータベース
 - アプリケーション開発フレームワークとは
 - データベース開発のフレームワーク (ORMマッピング)
 - SQLAlchemyによるデータベースアクセス

1. Pythonからのデータ作成 (INSERT)

Pythonからのデータ作成

sqlite3では、データの作成・削除なども行うことができる。

前回共有したSQLiteのsyllabusデータベースには、PostgreSQLのデータベースではなかった、教員テーブルが追加されている。以下のSQL文により作成されており、データは入っていない。

```
CREATE TABLE 教員 (  
    教員id integer PRIMARY KEY AUTOINCREMENT,  
    氏名 text,  
    年齢 integer  
);
```

- 教員idについて。SQLiteにはserial型がないために、integer型を用いており、AUTOINCREMENTのオプションを追加している。これは、新しい行が追加される毎に、自動的に1ずつ値が大きくなる（インクリメントする）指定である。

サンプルプログラム9-1：データの作成

以下のサンプルプログラムを実行してみよう。次のセルを実行し、教員にデータができているかを確認しよう

```
import sqlite3

connection = sqlite3.connect('syllabus.sqlite3')
cursor = connection.cursor()

sql = "INSERT INTO 教員 (氏名, 年齢) VALUES('鈴木源吾', 25)"
cursor.execute(sql)

connection.commit()

cursor.close()
connection.close()
```

サンプルプログラムの解説

```
sql = "INSERT INTO 教員 (氏名, 年齢) VALUES ('鈴木源吾', 25);"  
cursor.execute(sql)
```

INSERTのSQLの実行は、SELECT文のときと同じで、カーソルオブジェクトに対してexecute()を使えば良い

```
connection.commit()
```

これが、データベースの基礎で学んだ「コミット」である（トランザクションに関する回）。コミットを発行することによって、データがデータベースに反映される

コミットの復習：データベースの基礎 第10回

トランザクションのコミットとアボート

トランザクションは原子性を持つと説明した。含まれる複数のSQL文は、

- すべてを実行する
- すべてをなかったことにする

の2種類しかないということだった。それぞれをDBMSに行わせる命令を

- すべてを実行しろという命令→コミット(**commit**)
- すべてをなかったことにしろという命令→アボート (**abort**)

と呼ぶ。トランザクションをコミットする、アボートするという言い方をする

コミットしなかったら何が起こるのか？

commit()をコメントアウトした，以下のサンプルプログラム9-2のセルを実行した後，次のセルのSQL文を実行することで，テーブル 教員の中身を確認してみよう

```
# サンプルプログラム9-2
import sqlite3

connection = sqlite3.connect('syllabus.sqlite3')
cursor = connection.cursor()

sql = "INSERT INTO 教員 (氏名, 年齢) VALUES('柄沢直之', 24)"
cursor.execute(sql)

#connection.commit()

cursor.close()
connection.close()
```

コミットしなかったら何が起こるのか？

- commit()をコメントアウトすると、コミットされていないために、実行された結果が破棄されてしまう
- トランザクションを意識しなくてよいアプリケーションでは、いちいちコミットを発行せずに自動でコミットしてほしいときもある
- そのために、SQLが発行される毎にコミットされるようにするオプションを使用する。これは**自動コミット**（オートコミット）と呼ばれることが多い
- SQLiteでは、connect関数のオプションisolation_levelをNoneに設定することで自動コミットとなる
- isolationは、データベースの基礎第10回で学んだ、隔離性（分離性）である
- isolation_levelは隔離性水準・分離性レベル・分離レベルなどと呼ばれ、DBMSが提供する隔離性のレベルを示す値である（教科書p.318）。自動コミットを隔離性レベルの値として指定するのはSQLite独自（PostgreSQLでは別の指定方法）

自動コミット（オートコミット）

以下のサンプルプログラムを実行した後に、データの中身を確認してみよう

```
# サンプルプログラム9-3：自動コミット（オートコミット）

import sqlite3

connection = sqlite3.connect('syllabus.sqlite3', isolation_level=None)
cursor = connection.cursor()

sql = "INSERT INTO 教員（氏名，年齢）VALUES('柄沢直之', 24)"
cursor.execute(sql)

cursor.close()
connection.close()
```

INSERT文でのパラメータの利用

SELECT文と同様に，INSERT文でもSQL文にパラメータを用いることができる
パラメータの指定方法は2種類ある．ノートブックのサンプルを確認すること

サンプルプログラム9-4：INSERT文でのパラメータの利用

```
import sqlite3

connection = sqlite3.connect('syllabus.sqlite3')
cursor = connection.cursor()

cursor.execute("INSERT INTO 教員 (氏名, 年齢) VALUES (?, ?);", ("吉田貴裕", 23))

connection.commit()

cursor.close()
connection.close()
```

演習：第4回課題の第2問

以下を行うプログラムを作成せよ．ノートブックの演習セルを利用すること

- セルの先頭で氏名と年齢を変数（例：name, age）に代入し，
- syllabusデータベースの教員テーブルに対して，
- 氏名と年齢のカラムに指定された値を入れた，行を1つ作成する
- パラメータ化（?記号 または :名前 形式）を使うこと

例：name="田中一郎", age=26で実行すると，syllabusデータベースの教員テーブルに，氏名が「田中一郎」で，年齢が26である行が作成される

2. アプリケーション開発フレームワークとデータベース

アプリケーション開発フレームワーク

- アプリケーションをすべて自分で作成するのは大変
- アプリケーション開発フレームワークとは，アプリケーションを開発する上での様々な共通機能や，アプリケーションを生成する等の仕組みを含むソフトウェア
 - 以下，フレームワークと略して呼ぶ
- 重要な特徴は「**抽象化**」である．開発者は低レベルの処理を気にせずに，高レベルの概念や処理に集中でき，生産性や再利用性を大きく向上できる
- Pythonを開発言語とするのWebアプリケーション開発のフレームワークとしては，Flask，Djangoがある
- 今回学ぶ，SQLAlchemyはデータベース操作に関するフレームワークであり，DBMSの機能やSQLを抽象化することで，DBMSによる差異を隠蔽できる

データベースアクセスの抽象化とORマッピング

- フレームワークにおいて、データベースアクセスを抽象化する代表的な機能がORマッピング
- ORマッピング：オブジェクトとリレーション（テーブル）の変換機能
 - SQLを意識せずに、プログラム言語における（メモリ上の）クラスとオブジェクトのようにデータベースのデータにアクセスし参照・更新できる
 - 抽象化することにより、複雑なSQLを隠蔽できる
 - 一方、使い方を誤ると、性能劣化の原因となることもあるので注意が必要
 - ORマッピングを実現するプログラムやライブラリを**ORMapper**と呼ぶ
- フレームワークのデータベースアクセス機能としては、Ruby on Railsが使いやすいと評価が高く、他言語のフレームワークにも大きな影響を与えている

SQLAlchemyとは？

- Pythonから利用できるデータベース操作のフレームワーク
- 公式サイトによると、「The Python SQL Toolkit and Object Relational Mapper」
 - 公式サイト: <https://www.sqlalchemy.org>
 - 主要な機能は、ORマッピングと、SQLを抽象化して記述するAPIである
- Web開発フレームワークとの組合せでよく使用され、特にFlaskとの相性がよい
 - ライブラリflask_sqlalchemyはFlaskとよく統合されており、モデル作成（後述）をシンプルに実施できる
 - 一方、Djangoは独自のORMマッパーを持っている
- 今回はORMマッピングを中心に概要を紹介する
 - 日本語の解説書はないが、Flaskの本や記事の中で紹介されることがある
 - 英語の解説書としては、Essential SQLAlchemy (O'Reilly Media)がある

SQLAlchemyの基本

まずは、SQLをそのまま実行する使い方から入り、基本的な接続や検索の流れを理解しよう。以下のサンプルプログラム9-5を動かしてみよう
(対象学年が1であるような科目の行をすべて出力する)

```
# サンプルプログラム9-5 : SQLAlchemyの基本

from sqlalchemy import create_engine, text

engine = create_engine('sqlite:///syllabus.sqlite3')

with engine.connect() as connection:
    result = connection.execute(
        text("SELECT * FROM 授業科目 WHERE 対象学年 = :year"), {"year":1})
    for row in result:
        print(row)
```

サンプルの実行結果例

(1, '確率論', '確率の基礎を学ぶ', '課題, 試験', 1)
(2, 'DB基礎', 'DBの使い方を学ぶ', '試験', 1)

サンプルの説明：Engineオブジェクト

```
engine = create_engine('sqlite:///syllabus.sqlite3')
```

- SQLAlchemyでは、データベースへの接続やSQL文の実行を管理するEngineオブジェクトを利用する
- Engineオブジェクトは、create_engine()関数によって作成される
- 引数としてURL形式の接続文字列をとる．これはSQLiteデータベースにアクセスする例だが、PostgreSQLなどの他のDBMSも利用でき、基本的にはこの接続文字列を修正するだけで、他のDBMSにプログラムを移行できる
 - 例：プロトタイプではSQLiteを使い、実用化でPostgreSQLに移行する
- 上記は、カレントディレクトリにあるsyllabus.sqlite3というSQLiteのデータベースに接続するためのEngineオブジェクトを生成していることを意味する

サンプルの説明：接続とwith文

```
with engine.connect() as connection:
```

- Engineオブジェクトに対しconnect()関数を適用した返却値のConnectionオブジェクトを用いて、データベースへの接続を行う
- with文は後処理を自動で行ってほしいときに使える構文
 - 接続→DBアクセス→接続終了の一連の流れで、接続終了を自動で実行してくれる
 - 終了処理の忘れがなくなるので便利
 - ファイルの読み書きでもwith文は利用される（ファイルのクローズを自動化）

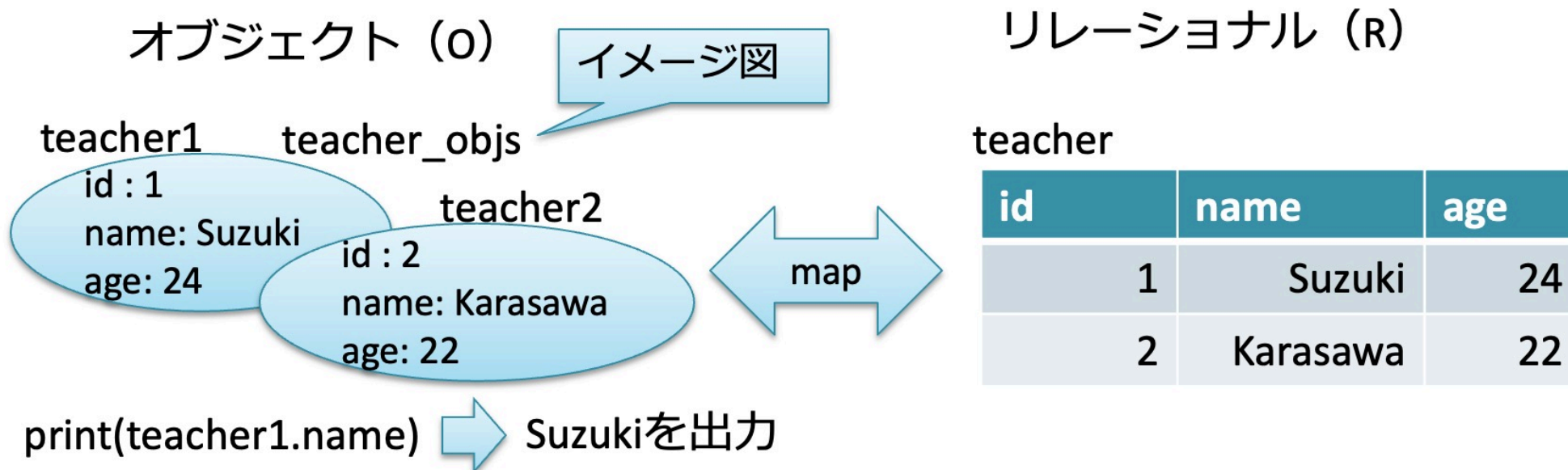
サンプルの説明

```
result = connection.execute(  
    text("SELECT * FROM 授業科目 WHERE 対象学年 = :year"), {"year":1})
```

- SQL文を実行する
- :yearは、SQL文でパラメータを使うSQLAlchemyの構文である。次の引数の{"year":1}は、yearの値を1とすることを表す（Pythonの辞書型オブジェクト）
 - どのDBMSでもこの形式が使える（sqlite3ライブラリだと?形式だった）
- 注意：execute関数の引数に直接SQL文字列を指定するとエラーとなる。
SQLAlchemyの関数であるtext関数によって、実行可能なオブジェクトに変換して渡す必要がある
- 第8回で説明した**SQLインジェクション対策**は SQLAlchemy でも同様に有効。
:year 形式のパラメータを使うこと（文字列連結でSQL文を組み立てないこと）

ORMapping超入門

SQLAlchemyのORMapping機能を用いて，syllabusデータベースにteacherというテーブルを作成し，それをオブジェクトとして作成したり，参照してみよう（次ページ以降に5つのサンプルプログラム．モデルの定義：9-6，テーブル作成：9-7，データ作成：9-8，データ検索：9-9，9-10）



サンプルプログラム：モデルを定義

サンプルプログラム9-6. セルを実行するとmodel.pyに下のプログラムが書き込まれる

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

# モデルベースクラスの作成（空のクラス）
class Base(DeclarativeBase):
    pass

# Baseクラスを継承したTeacherクラスを定義する
class Teacher(Base):
    # テーブル名
    __tablename__ = 'teacher'

    # カラムを定義（型ヒントで型を指定）
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column()
    age: Mapped[int] = mapped_column()
```


解説：モデルベースクラスとモデルクラス

- ORマッピングに使うオブジェクトのクラスを「モデルクラス」と呼ぶ
 - 参考：「モデル」という用語は，MVC（Model-View-Controller）というソフトウェア設計モデルからきている
- 「モデルクラス」は，「モデルベースクラス」を継承し，カスタマイズすることで作成できる
- 「モデルベースクラス」は，SQLAlchemy 2.0系では `DeclarativeBase` を継承した空のクラスとして定義する

```
class Base(DeclarativeBase):  
    pass
```

↓ Baseというモデルベースクラスを継承し，Teacherというモデルクラスを定義する

```
class Teacher(Base):
```

解説：モデルクラスの定義

- クラス定義の中で、テーブル名やカラムを定義していく
- テーブル名として、「teacher」を使う
- 3つのカラム (id, name, age) をPythonの型ヒント (`Mapped[int]`, `Mapped[str]`) と `mapped_column()` で定義している
 - `Mapped[int]` → SQLのINTEGER型に対応
 - `Mapped[str]` → SQLのVARCHAR型に対応
- この定義は、DBMSから独立である (SQLiteでもPostgreSQLでも同じ)

```
class Teacher(Base):  
    __tablename__ = 'teacher'  
  
    id: Mapped[int] = mapped_column(primary_key=True)  
    name: Mapped[str] = mapped_column()  
    age: Mapped[int] = mapped_column()
```

サンプルプログラム：テーブルを作成

サンプルプログラム9-7. このセルを実行するとteacherテーブルが作成される

```
from sqlalchemy import create_engine
from model import Base

# データベース（テーブル）作成
engine=create_engine('sqlite:///syllabus.sqlite3')
meta = Base.metadata
meta.create_all(engine)
```

以下の箇所で、前のセルを実行することで作成されたmodel.pyで定義したBaseクラスをインポートし利用可能にしている

```
from model import Base
```

解説：「メタデータ」からテーブル生成

- モデルクラスを作成することによって、「メタデータ」（データの定義に関するデータ）が作成されている
- メタデータは，Base.metadataで得られる
- create_all()関数を用いて，エンジンとメタデータを紐付けて，データベースの中にテーブル（teacher）の実体が作られる

```
engine=create_engine('sqlite:///syllabus.sqlite3')  
meta = Base.metadata  
meta.create_all(engine)
```

- 次のセルを実行し，実際にテーブルが作られたか確認しよう

サンプルプログラム：データの作成

サンプルプログラム9-8（実際のプログラムでは3人のデータを追加）

```
from model import Teacher
from sqlalchemy import create_engine
from sqlalchemy.orm import Session

# エンジン生成
engine = create_engine("sqlite:///syllabus.sqlite3")

# セッションをwith文で生成（終了時に自動でクローズ）
with Session(engine) as session:
    # データ追加
    t_obj = Teacher(name="Suzuki", age=25)
    session.add(t_obj)
    （中略：2人のデータを追加）
    session.commit()
```

解説：セッション

- 1行目で、model.pyからTeacherクラスをインポートし利用可能にしている
- 「セッション」はエンジン（のデータベース）に対するトランザクションに対応する概念
 - セッションに対してcommit()を発行する
- SQLAlchemy 2.0系では、`Session` クラスをwith文で使うのが推奨形式
 - 終了処理（接続のクローズ）が自動で行われる
 - サンプル9-5で見た `with engine.connect() as connection:` と同じパターン

```
with Session(engine) as session:  
    # ここでデータ操作を行う  
    ...
```

解説：データ作成

- データを作成するためには、モデルクラスのオブジェクトを以下のように作成して

```
t_obj = Teacher(name="Suzuki", age=25)
```

- セッションに対して、add()関数を使うことで、データが実際に作成される最後にcommit()しないと、データは永続化されない
- このプログラムを実行した後に、実際にデータが作成されているかを確認しよう

```
session.add(t_obj)  
session.commit()
```

サンプルプログラム：データ検索（全件検索）

サンプルプログラム9-9：実行すると前に作成したデータが表示される

```
from model import Teacher
from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session

# エンジン生成
engine = create_engine("sqlite:///syllabus.sqlite3")

with Session(engine) as session:
    # 検索：select文を作成し、scalars()でTeacherオブジェクトを取り出す
    stmt = select(Teacher)
    teachers = session.scalars(stmt).all()
    for teacher in teachers:
        print(teacher.name)
```


解説：データ検索（全件検索）

```
stmt = select(Teacher)
teachers = session.scalars(stmt).all()
```

- SQLAlchemy 2.0系では、`select()` 関数でクエリを組み立てる
 - `select(Teacher)` は、Teacherテーブルからの SELECT 文に対応
- セッションの `scalars()` で実行すると、Teacherオブジェクトの集まりが得られる
 - `.all()` で全件をリストとして取得
- 結果のオブジェクトのカラムは、`teacher.name` のようにオブジェクトの属性として参照できる

サンプルプログラム：データ検索（条件検索）

サンプルプログラム9-10： `where()` を使うと条件検索ができる

この例では、年齢（age）が24以下となる条件（24歳以下）をつけている

```
from model import Teacher
from sqlalchemy import create_engine, select
from sqlalchemy.orm import Session

# エンジン生成
engine = create_engine("sqlite:///syllabus.sqlite3")

with Session(engine) as session:
    # 検索：where() で条件を指定
    stmt = select(Teacher).where(Teacher.age <= 24)
    teachers = session.scalars(stmt).all()
    for teacher in teachers:
        print(teacher.name)
```

演習：第4回課題の第3問

以下を行うプログラムをSQLAlchemyの**ORMマッピング機能を用いて**作成せよ（SQL文を直接実行する方法は不可．第4回課題の第2問と同等の処理を，ORMマッピングで実現する）

- セルの先頭で氏名と年齢を変数（例：name, age）に代入し，
- syllabusデータベースのteacherテーブルに対して，
- nameとageに指定された値を入れた，行を作成する

例：name="田中一郎", age=26で実行すると，syllabusデータベースのteacherテーブルに，nameが「田中一郎」で，ageが26である行が作成される

演習の提出について

- 第8回・第9回の演習ノートブックをWebClassでリンク提出すること